Statistique inférentielle

Chapitre 21 : Mise en œuvre sur R et Python

Jaouad Madkour

31 octobre 2026

Plan du chapitre

Le programme **2**
Refaire le cours sur machine . . . 2
Ce qui a été exécuté . . . 3

Sur R : les données et les lois **3**
Charger et tirer l'échantillon . . . 3
La population et l'échantillon . . . 3
Les lois du cours . . . 4
La distribution de la moyenne, par simulation . . . 4
Le résultat . . . 5

Sur R : intervalles et tests **5**
L'intervalle de confiance d'une moyenne . . . 5
Une proportion . . . 5
Les réglages par défaut de prop.test . . . 6
L'intervalle d'une variance . . . 7
La puissance et la taille d'échantillon . . . 7
Le test d'une moyenne . . . 8
Lire la sortie de t.test . . . 9
Deux groupes . . . 9
Données appariées . . . 9
L'analyse de la variance . . . 10
Les comparaisons deux à deux . . . 10
Le khi-deux d'indépendance . . . 10
Le khi-deux d'adéquation, et un piège . . . 11
La régression . . . 11
Chaque nombre a été calculé à la main . . . 12
Intervalles et prévisions . . . 12
Le nuage et les résidus . . . 12
La couverture des intervalles . . . 13

Sur Python **13**
Les mêmes données . . . 13

Les lois et le test d'une moyenne . . . 14
Une proportion : le piège du dénominateur . . . 14
La puissance . . . 14
Deux groupes . . . 15
L'analyse de la variance . . . 15
Le khi-deux . . . 15
La régression . . . 16
Les prévisions . . . 16
La simulation du chapitre 13, reproduite à l'identique . . . 16
Le résultat, en image . . . 17
Les deux bandes, en Python . . . 17

Les pièges et la correspondance **18**
Les réglages par défaut qui changent le résultat . . . 18
La correspondance des fonctions . . . 18

S'entraîner **18**
Exercice 1 : lire une sortie R . . . 18
Exercice 1 : solution . . . 19
Exercice 2 : corriger un script Python . . . 19
Exercice 2 : solution . . . 19
Exercice 3 : écrire le code . . . 19
Exercice 3 : solution . . . 20

Ce qu'il faut retenir **20**
Le résumé du chapitre . . . 20
La fin du parcours . . . 20

Le programme

Refaire le cours sur machine

Ce que ce chapitre reprend

Ce qu'on refait	Chapitres
Lois d'échantillonnage, par simulation et par les quantiles	2 à 4
Intervalles de confiance, taille d'échantillon	8 à 11
Tests d'une moyenne, d'une proportion, puissance	12 à 15
Deux groupes, ANOVA, khi-deux	16 à 18
Régression, test d'un coefficient, prévision	19 et 20

Le but n'est pas d'apprendre un logiciel

C'est de **vérifier** que chaque nombre calculé à la main est bien celui qu'affiche la machine — et de voir qu'une fonction en remplace une page.
Comprendre la formule reste indispensable : c'est ce qui permet de choisir la bonne fonction et de reconnaître une sortie aberrante.

Ce qui a été exécuté

Les versions utilisées

Logiciel	Version
R	4.3.3 (fonctions de base : `stats`)
Python	3.13, avec `pandas` 3.0, `numpy` 2.5, `scipy` 1.18, `statsmodels` 0.15

Toutes les sorties sont réelles

Chaque bloc de code de ce chapitre a été exécuté sur `salaires.csv`, et les sorties sont recopiées telles quelles, à quelques lignes près.
Faites-les tourner vous-même : un écart avec ces sorties signale une erreur de saisie, ou une version différente.

Sur R : les données et les lois

Charger et tirer l'échantillon

```
sal <- read.csv("salaires.csv", stringsAsFactors = TRUE)
ech <- subset(sal, id %% 10 == 0)   # un identifiant sur dix
str(ech)
```

La sortie

```
'data.frame':   50 obs. of  6 variables:
 $ id        : int  10 20 30 40 50 60 70 80 90 100 ...
 $ salaire   : int  7150 5510 10070 5390 9470 7610 13600 ...
 $ anciennete: int  7 5 13 6 4 13 20 8 2 10 ...
 $ diplome   : Factor w/ 3 levels "Bac","Bac+2",..: 3 3 3 ...
 $ region    : Factor w/ 4 levels "Autres","Casablanca",..: 2 ...
 $ sexe      : Factor w/ 2 levels "F","H": 2 1 2 2 2 1 2 2 1 1 ...
```

La population et l'échantillon

```
mean(sal$salaire)                  # 7158.26   m
sd(sal$salaire) * sqrt(499/500)    # 2598.1    sigma (population)
mean(ech$salaire)                  # 7005.8    moyenne d'échantillon
sd(ech$salaire)                    # 2250.976  s, en n - 1
```

Deux écarts-types, deux touches

`sd()` divise par $n-1$: c'est l'estimateur du chapitre 5, celui qu'on emploie sur un échantillon. Pour l'écart-type d'une population complète, il faut multiplier par $\sqrt{(n-1)/n}$: R ne propose que la première formule, parce que c'est elle dont l'inférence a besoin.

Démonstration : Pourquoi l'écart entre les deux écarts-types est de 1 pour cent à n égal 50

Notons $\hat{\sigma}^2 = \frac{1}{n}\sum_i (x_i - \bar{x})^2$ (diviseur n, celui de `numpy.std()`) et $s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$ (diviseur $n-1$, celui de `sd()` et de `pandas.std()`). Les deux sommes sont les mêmes, donc

$$s^2 = \frac{n}{n-1}\,\hat{\sigma}^2, \qquad s = \hat{\sigma}\sqrt{\frac{n}{n-1}}, \qquad \frac{s}{\hat{\sigma}} - 1 = \sqrt{\frac{n}{n-1}} - 1 \approx \frac{1}{2(n-1)}.$$

Le rapport est 1,0102 pour $n = 50$ (1 %) et 1,0541 pour $n = 10$ (5 %).

Sur nos données. `ech.salaire.std()` donne 2 250,976 et `np.std(ech.salaire)` donne $2\,250{,}976 \times \sqrt{49/50} = 2\,228{,}353$. Pour la population de $N = 500$ salariés, `sd()` renvoie 2 600,70, le diviseur étant $N - 1$; l'écart-type σ du cours, qui divise par N, est $2\,600{,}70 \times \sqrt{499/500} = 2\,598{,}10$.

Ce que cela dit. Le diviseur $n - 1$ est celui du chapitre 5 : il rend s^2 sans biais pour σ^2. Sur un échantillon on veut s ; sur une population complète, qui n'a rien à estimer, on veut σ avec le diviseur N. Les deux outils font chacun l'un des deux calculs par défaut, d'où l'erreur la plus fréquente du passage de R à Python.

■

Les lois du cours

```r
qnorm(0.975)                          # 1.959964
qt(c(0.95, 0.975), df = 49)           # 1.676551 2.009575
qchisq(0.95, df = 2)                  # 5.991465
qf(0.95, df1 = 2, df2 = 47)           # 3.195056
pt(1.589, df = 49, lower.tail = FALSE) # 0.05924637
```

Une famille de fonctions par loi

Préfixe	Calcule	Exemple
`q`	un quantile (la table, à l'envers)	`qt(0.975, 49)` donne 2,010
`p`	une probabilité cumulée	`pt(1.589, 49)` donne 0,941
`d`	une densité (pour tracer)	`dnorm(x)`
`r`	des tirages aléatoires	`rnorm(10)`

La distribution de la moyenne, par simulation

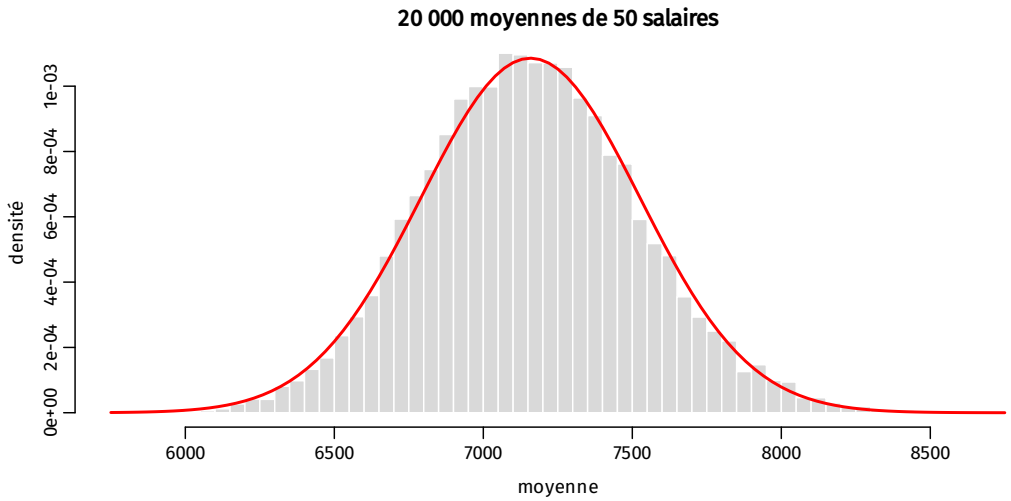
```r
set.seed(20261022)
moy <- replicate(20000,
                 mean(sample(sal$salaire, 50, replace = TRUE)))
mean(moy)                             # 7157.199   ->  m = 7158.26
sd(moy)                               # 365.0535   ->  2598.1 / sqrt(50)
2598.1 / sqrt(50)                     # 367.4268
hist(moy, breaks = 50, freq = FALSE)
```

```r
curve(dnorm(x, 7158.26, 2598.10 / sqrt(50)), add = TRUE)
```

Cinq lignes pour le chapitre 2

La moyenne des moyennes tombe sur m, leur écart-type sur $\sigma/\sqrt{n}$, et l'histogramme épouse la cloche alors que les salaires ne sont pas normaux.

Le résultat



Sur R : intervalles et tests

L'intervalle de confiance d'une moyenne

```r
t.test(ech$salaire)$conf.int       # 6366.08 7645.52
n <- 50; xb <- mean(ech$salaire); s <- sd(ech$salaire)
xb + c(-1, 1) * qt(0.975, n - 1) * s / sqrt(n)   # idem
```

Pourquoi pas exactement le chapitre 9

Le chapitre 9 donnait [6 365,94 ; 7 645,66] avec le quantile de la table, arrondi à 2,010. R utilise 2,009575 : l'intervalle est plus étroit de 0,14 dirham de chaque côté.
Un écart de cet ordre entre la main et la machine est **normal** ; un écart de cent dirhams signale une erreur.

Une proportion

```r
prop.test(13, 50, p = 0.15, correct = FALSE)
```

La sortie, abrégée

```
X-squared = 4.7451, df = 1, p-value = 0.02938
95 percent confidence interval:
 0.1587153 0.3955316
```

Trois résultats du cours d'un coup

`X-squared` est le $z^2 = 2{,}178^2$ du chapitre 15, et la p-value est la même, 0,029.
L'intervalle est celui de **Wilson** du chapitre 10, [0,159 ; 0,396] : `prop.test` inverse le test du score, exactement comme le chapitre 15 l'a montré.

Les réglages par défaut de prop.test

```
prop.test(13, 50, p = 0.15)$p.value   # 0.04767038  (correction)
binom.test(13, 50, p = 0.15)$p.value  # 0.04424905  (test exact)
```

Un détail qui change le résultat

Sans `correct = FALSE`, `prop.test()` applique une **correction de continuité** : la p-value passe de 0,029 à 0,048.
`binom.test()` donne le test exact du chapitre 15, 0,044. Les trois sont défendables ; ce qui ne l'est pas, c'est de ne pas savoir lequel on emploie.

Démonstration : Quatre réglages, quatre p-values pour les treize Bac+5

Avec $n = 50$, 13 succès, $p_0 = 0{,}15$ et $f = 0{,}26$, voici ce que calcule chaque réglage.

réglage	calcul	z ou χ^2	p
Wald (`proportions_ztest()` par défaut)	$\dfrac{0{,}11}{\sqrt{0{,}26 \times 0{,}74/50}}$	$z = 1{,}773$	0,0762
score (`prop.test(correct = FALSE)`, ou `prop_var=0.15`)	$\dfrac{0{,}11}{\sqrt{0{,}15 \times 0{,}85/50}}$	$z = 2{,}178$, $\chi^2 = 4{,}745$	0,0294
score avec correction de continuité (`prop.test()` par défaut)	$\dfrac{0{,}11 - 0{,}01}{\sqrt{0{,}15 \times 0{,}85/50}}$	$z = 1{,}980$, $\chi^2 = 3{,}922$	0,0477
exact (`binom.test()`, `binomtest()`)	somme des $P(X = k) \leq P(X = 13)$		0,0442

La correction de continuité. Le nombre de succès est entier alors que la normale est continue : on retranche à $|f - p_0|$ la demi-unité $\frac{1}{2n} = 0{,}01$ (une demi-observation sur 50). C'est le calcul de la démonstration du chapitre 15, qui explique aussi pourquoi l'exact se situe entre les deux.

Le dénominateur de Wald. Il utilise $f = 0{,}26$ au lieu de $p_0 = 0{,}15$: l'écart-type est 0,0620 au lieu de 0,0505, d'où z plus petit. C'est la figure des deux dénominateurs du chapitre 15, et le niveau exact de ce test est de 5,9 %, non de 5 %.

Ce que cela dit. Les quatre p-values sont des calculs précis, chacun avec sa justification : seul le test de Wald donnerait un verdict différent à 5 %, et c'est celui dont le niveau réel s'écarte le plus de 5 %. Savoir lequel on lance, c'est savoir quelle option du logiciel retire ou garde une correction.

■

L'intervalle d'une variance

```r
bornes <- (n - 1) * s^2 / qchisq(c(0.975, 0.025), n - 1)
bornes                  # 3535592 7868118     (pour sigma^2)
sqrt(bornes)            # 1880.317 2805.017   (pour sigma)
```

Pas de fonction toute faite

R de base n'a pas de fonction pour l'intervalle d'une variance : on écrit la formule du chapitre 10, avec les quantiles **croisés** du khi-deux. On retrouve [1 880,3 ; 2 805,0].

La puissance et la taille d'échantillon

```r
power.t.test(n = 50, delta = 658.26, sd = 2598.10,
             type = "one.sample",
             alternative = "one.sided")$power     # 0.5485003
power.t.test(delta = 658.26, sd = 2598.10, power = 0.80,
             type = "one.sample",
             alternative = "one.sided")$n         # 97.68013
```

Le chapitre 13, en exact

`power.t.test()` calcule la puissance exacte par la loi de Student non centrale : 0,549, la valeur que la simulation du chapitre 13 avait trouvée.
Pour 80 % de puissance, il faut 98 observations, là où l'approximation normale du chapitre 13 en donnait 97.

Démonstration : La puissance exacte du test de Student, loi non centrale

Si la vraie moyenne est $m = m_0 + \delta$, la statistique du test s'écrit

$$T = \frac{\overline{X} - m_0}{S/\sqrt{n}} = \frac{Z + \lambda}{\sqrt{V/\nu}}, \qquad Z = \frac{\overline{X} - m}{\sigma/\sqrt{n}} \sim \mathcal{N}(0\,;1), \quad V = \frac{\nu S^2}{\sigma^2} \sim \chi^2_\nu, \quad \lambda = \frac{\delta\sqrt{n}}{\sigma},$$

avec $\nu = n - 1$, Z et V indépendants (chapitre 4). Sous H_0, $\lambda = 0$ et l'on retrouve Student

; sous H_1, la loi est celle de Student **non centrale** (paramètre λ). La puissance du test unilatéral est $P(T > t_{1-\alpha\,;\nu})$; en conditionnant sur V,

$$\pi = \int_0^\infty \left[1 - \Phi\left(t_{1-\alpha\,;\nu}\sqrt{v/\nu} - \lambda\right)\right] f_{\chi^2_\nu}(v)\, dv,$$

où $f_{\chi^2_\nu}$ est la densité du khi-deux. L'approximation normale du chapitre 13 revient à fixer $V = \nu$, c'est-à-dire à remplacer S par σ : $\pi \approx 1 - \Phi(z_{1-\alpha} - \lambda)$.

Nos chiffres ($\delta = 658{,}26$, $\sigma = 2\,598{,}10$, $\alpha = 5\,\%$) :

n	λ	approximation normale	formule exacte
50	1,792	0,558	0,5485
97	2,495		0,7975
98	2,508		0,8012
100	2,534		0,8082

La puissance de 80 % est atteinte pour $n = 97{,}68$ (R accepte un n non entier) : il faut donc 98 observations avec Student, contre 97 avec l'approximation normale.

Ce que cela dit. Le logiciel ne fait rien de plus que le calcul du chapitre 13 : il ne remplace S par σ qu'en moyenne, et non d'un coup. Cette fluctuation de S rend la loi de T plus dispersée que la normale et la puissance un peu plus basse (0,5485 au lieu de 0,558), d'où une observation de plus.

■

Le test d'une moyenne

```
t.test(ech$salaire, mu = 6500, alternative = "greater")
```

La sortie

```
    One Sample t-test

data:  ech$salaire
t = 1.5889, df = 49, p-value = 0.05926
alternative hypothesis: true mean is greater than 6500
95 percent confidence interval:
 6472.093      Inf
sample estimates:
mean of x
   7005.8
```

Lire la sortie de t.test

Les cinq étapes du chapitre 12, en une ligne

Ligne de la sortie	Chapitre
`alternative hypothesis: ... greater`	12 : la forme de H_1
`t = 1.5889, df = 49`	12 : la statistique et sa loi
`p-value = 0.05926`	14 : la p-value
`6472.093  Inf`	15 : la borne basse, équivalente au test

L'argument alternative

`alternative` vaut `"two.sided"` par défaut, `"greater"` ou `"less"` pour un test unilatéral. Avec un test unilatéral, R donne un intervalle **unilatéral** : la borne basse du chapitre 15.

Deux groupes

```r
t.test(salaire ~ sexe, data = ech)
```

La sortie, abrégée

```
    Welch Two Sample t-test

t = -0.83391, df = 46.855, p-value = 0.4086
95 percent confidence interval:
 -1810.9003   749.6016
```

R calcule F moins H

Les groupes sont pris dans l'ordre alphabétique : l'écart est « femmes moins hommes » (6 709 — 7 239), d'où le signe moins. C'est le test de Welch du chapitre 16, par défaut ; `var.equal = TRUE` donne la version à variances égales ($p = 0{,}4136$).

Données appariées

```r
avant <- c(52, 61, 45, 70, 58, 66, 49, 63)
apres <- c(55, 64, 46, 74, 60, 70, 50, 67)
t.test(apres, avant, paired = TRUE)
```

La sortie, abrégée

```
    Paired t-test

t = 6.0685, df = 7, p-value = 0.0005066
95 percent confidence interval:
 1.678439 3.821561
sample estimates:
mean difference
           2.75
```

L'analyse de la variance

```
summary(aov(salaire ~ diplome, data = ech))
oneway.test(salaire ~ diplome, data = ech)    # version de Welch
```

La sortie, abrégée

```
             Df    Sum Sq  Mean Sq F value Pr(>F)
diplome       2  37946073 18973037    4.24 0.0203 *
Residuals    47 210331745  4475144

F = 3.5088, num df = 2.000, denom df = 28.204, p-value = 0.04357
```

Le tableau du chapitre 17, à l'unité près

SCE, SCR, carrés moyens, $F = 4{,}24$ et $p = 0{,}020$; puis l'ANOVA de Welch, $F = 3{,}51$ et $p = 0{,}044$.

Les comparaisons deux à deux

```
pairwise.t.test(ech$salaire, ech$diplome,
                p.adjust.method = "bonferroni", pool.sd = FALSE)
```

La sortie

```
      Bac  Bac+2
Bac+2 0.98 -
Bac+5 0.04 0.20

P value adjustment method: bonferroni
```

Les p-values déjà corrigées

R affiche les p-values multipliées par trois (et plafonnées à 1) : on les compare directement à 0,05. Seul l'écart Bac / Bac+5 reste significatif, comme au chapitre 17.

Le khi-deux d'indépendance

```
tab <- table(ech$diplome, ech$sexe)
chisq.test(tab)
round(chisq.test(tab)$expected, 2)
```

La sortie, abrégée

```
X-squared = 0.23451, df = 2, p-value = 0.8894

           F     H
  Bac   8.80 11.20
  Bac+2 7.48  9.52
  Bac+5 5.72  7.28
```

Toujours afficher les attendus

`$expected` donne les effectifs attendus du chapitre 18 : c'est sur eux que porte la condition « au moins 5 ». R affiche un avertissement quand elle n'est pas remplie.

Le khi-deux d'adéquation, et un piège

```
pr <- prop.table(table(sal$region))    # parts dans la population
chisq.test(table(ech$region), p = pr)
#   X-squared = 0.37043, df = 3, p-value = 0.9463
t2 <- table(ech$diplome == "Bac+5", ech$sexe)
chisq.test(t2)$statistic                    # 0.02041877
chisq.test(t2, correct = FALSE)$statistic   # 0.2187002
```

La correction de Yates, sans prévenir

Pour un tableau 2×2, `chisq.test()` applique par défaut la **correction de Yates** : $\chi^2 = 0{,}020$ au lieu de $0{,}219$.
La valeur du chapitre 18, égale au z^2 du chapitre 16, s'obtient avec `correct = FALSE`.

Démonstration : La correction de Yates d'un tableau deux sur deux

La correction de Yates remplace chaque écart $|O-E|$ par $|O-E|-\frac{1}{2}$, la même demi-unité que la correction de continuité d'une proportion : $\chi^2_{\text{Yates}} = \sum(|O-E|-\frac{1}{2})^2/E$. Dans un tableau 2×2 de cases a, b, c, d et de total n, les quatre écarts valent $|ad-bc|/n$ (démonstration du chapitre 18), donc

$$\chi^2_{\text{Yates}} = \Big(\frac{|ad-bc|}{n} - \frac{1}{2}\Big)^2 \sum_{ij} \frac{1}{E_{ij}} = \frac{n\,(|ad-bc| - \frac{n}{2})^2}{n_{1\bullet}\, n_{2\bullet}\, n_{\bullet 1}\, n_{\bullet 2}}.$$

Bac+5 selon le sexe. Avec $a = 5$, $b = 8$, $c = 17$, $d = 20$ et $n = 50$: $|ad-bc| = 36$ et $\frac{n}{2} = 25$, donc

$$\chi^2_{\text{Yates}} = \frac{50 \times (36-25)^2}{13 \times 37 \times 22 \times 28} = \frac{50 \times 121}{296\,296} = 0{,}0204, \qquad p = 0{,}886,$$

contre $\chi^2 = 50 \times 36^2/296\,296 = 0{,}2187$ sans la correction.

Ce que cela dit. La correction retire 25 à un numérateur qui ne vaut que 36 : elle en supprime plus des deux tiers, et la statistique est divisée par plus de dix. Elle protège quand les effectifs sont petits (c'est un test conservateur), mais elle empêche de retrouver le z^2 du chapitre 16 ; c'est pourquoi le cours demande `correct = FALSE`.

■

La régression

```
mod <- lm(salaire ~ anciennete, data = ech)
summary(mod)
```

La sortie, abrégée

```
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  4288.72     468.58   9.153 4.23e-12 ***
anciennete    263.79      39.54   6.671 2.35e-08 ***

Residual standard error: 1638 on 48 degrees of freedom
Multiple R-squared:  0.4811,    Adjusted R-squared:  0.4703
F-statistic:  44.5 on 1 and 48 DF,  p-value: 2.35e-08
```

Chaque nombre a été calculé à la main

Où les retrouver

Nombre de la sortie	Valeur	Chapitre
`Estimate`	4 288,72 et 263,79	19
`Std. Error`, `t value`	39,54 et 6,67	20
`Residual standard error`	1 638 ($n - 2 = 48$)	20
`R-squared`, `F-statistic`	0,481 et $44{,}5 = 6{,}671^2$	20

Le logiciel n'ajoute rien

Il calcule plus vite. Seul nombre nouveau : le R^2 « ajusté », qui pénalise le nombre de variables explicatives ; il prend son sens en régression multiple.

Intervalles et prévisions

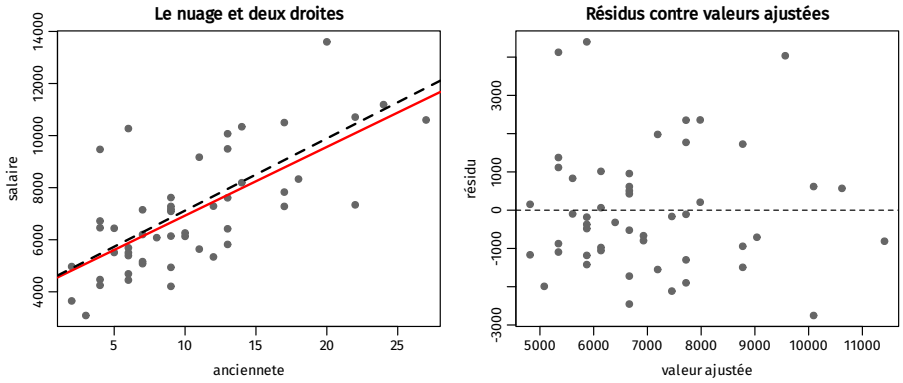
```r
confint(mod)
predict(mod, data.frame(anciennete = 10), interval = "confidence")
predict(mod, data.frame(anciennete = 10), interval = "prediction")
```

La sortie

```
                2.5 %    97.5 %
(Intercept) 3346.5786 5230.8578
anciennete   184.2882  343.3005
       fit      lwr      upr
1 6926.662 6460.211 7393.113
       fit      lwr      upr
1 6926.662 3599.806 10253.52
```

Le nuage et les résidus

```r
plot(salaire ~ anciennete, data = ech)
abline(mod)                              # la droite estimée
abline(4347.23, 277.17, lty = 2)         # la vraie droite
plot(fitted(mod), resid(mod)); abline(h = 0, lty = 2)
```



La couverture des intervalles

```r
set.seed(20261022)
couvre <- replicate(10000, {
  e  <- sample(sal$salaire, 50, replace = TRUE)
  ic <- t.test(e)$conf.int
  ic[1] <= 7158.26 & 7158.26 <= ic[2]
})
mean(couvre)        # 0.941
```

La démonstration la plus convaincante du cours

Sur dix mille échantillons, 94,1 % des intervalles contiennent la vraie moyenne — un peu moins de 95 %, à cause de l'asymétrie des salaires (chapitre 9). **Faites tourner ce bloc : il vaut trois heures d'explication.**

Sur Python

Les mêmes données

```python
import numpy as np, pandas as pd
from scipy import stats

sal = pd.read_csv("salaires.csv")
ech = sal[sal.id % 10 == 0]

ech.salaire.mean()            # 7005.8
ech.salaire.std()             # 2250.97628705812   (ddof=1)
np.std(ech.salaire)           # 2228.3528356164784 (ddof=0)
sal.salaire.std(ddof=0)       # 2598.0996848465993 (sigma)
```

Deux conventions opposées, à connaître

`pandas.std()` divise par $n-1$; `numpy.std()` divise par n. L'écart vaut 1 % à $n = 50$, 5 % à $n = 10$. **C'est la source d'erreur la plus fréquente du passage de R à Python.**

Les lois et le test d'une moyenne

```
stats.norm.ppf(0.975)              # 1.959963984540054
stats.t.ppf([0.95, 0.975], 49)     # [1.67655089 2.00957524]
stats.chi2.ppf(0.95, 2)            # 5.991464547107979
stats.f.ppf(0.95, 2, 47)           # 3.1950562807372145

r = stats.ttest_1samp(ech.salaire, 6500, alternative="greater")
r.statistic, r.pvalue              # 1.5888866176888397 0.0592591...
r.confidence_interval(0.95)        # low=6472.0932949117, high=inf
```

Les préfixes de R deviennent des méthodes

`qt` devient `stats.t.ppf`, `pt` devient `stats.t.cdf` (et `stats.t.sf` pour la queue droite), `dt` devient `stats.t.pdf`. Sans `alternative`, `ttest_1samp` est bilatéral et son intervalle est [6 366,08 ; 7 645,52], comme en R.

Une proportion : le piège du dénominateur

```
from statsmodels.stats.proportion import proportions_ztest
from statsmodels.stats.proportion import proportion_confint
proportions_ztest(13, 50, value=0.15)          # (1.7733, 0.0762)
proportions_ztest(13, 50, value=0.15,
                  prop_var=0.15)               # (2.1783, 0.0294)
proportion_confint(13, 50, method="wilson")    # (0.1587, 0.3955)
stats.binomtest(13, 50, 0.15).pvalue           # 0.04424904...
```

Par défaut, f au dénominateur

`proportions_ztest()` utilise par défaut la proportion **observée** au dénominateur : c'est le test de Wald, qui ne rejette pas ($p = 0{,}076$).
Le test du chapitre 15, avec p_0 au dénominateur, exige `prop_var=0.15`. C'est exactement la figure des deux dénominateurs, en une option.

La puissance

```
from statsmodels.stats.power import TTestPower
d = 658.26 / 2598.10                    # écart en écarts-types
TTestPower().power(effect_size=d, nobs=50, alpha=0.05,
                   alternative="larger")          # 0.5485
TTestPower().solve_power(effect_size=d, power=0.8, alpha=0.05,
                         alternative="larger")    # 97.68
```

Un écart réduit

`statsmodels` attend l'écart **en écarts-types**, $\delta/\sigma = 0{,}253$, là où R prend `delta` et `sd` séparément. Les résultats sont identiques : 0,5485 et 97,68.

Deux groupes

```
h = ech[ech.sexe == "H"].salaire
f = ech[ech.sexe == "F"].salaire
stats.ttest_ind(h, f)                    # t=0.8247, p=0.4136
stats.ttest_ind(h, f, equal_var=False)   # t=0.8339, p=0.4086
avant = np.array([52, 61, 45, 70, 58, 66, 49, 63])
apres = np.array([55, 64, 46, 74, 60, 70, 50, 67])
stats.ttest_rel(apres, avant)            # t=6.0685, p=0.0005
```

Le défaut inverse de celui de R

`stats.ttest_ind()` suppose par défaut des variances **égales** ; `t.test()` de R fait Welch. Le chapitre 16 recommande Welch : en Python, il faut écrire `equal_var=False`.

L'analyse de la variance

```
groupes = [g.salaire.to_numpy()
           for _, g in ech.groupby("diplome")]
stats.f_oneway(*groupes)        # F=4.2396, p=0.020287

from statsmodels.stats.oneway import anova_oneway
anova_oneway(groupes, use_var="unequal").pvalue   # 0.04357

from statsmodels.stats.multitest import multipletests
multipletests(p, method="bonferroni")[1]   # [0.985 0.040 0.202]
```

Les mêmes nombres qu'en R

$F = 4{,}24$, Welch $p = 0{,}044$, et les trois p-values corrigées du chapitre 17 ; `p` est la liste des trois p-values de Welch, calculées paire par paire avec `ttest_ind`.

Le khi-deux

```
tab = pd.crosstab(ech.diplome, ech.sexe)
res = stats.chi2_contingency(tab)
res.statistic, res.pvalue, res.dof   # 0.2345081389 0.889359 2
res.expected_freq        # [[8.8 11.2] [7.48 9.52] [5.72 7.28]]
obs = ech.region.value_counts().sort_index()
pr = sal.region.value_counts(normalize=True).sort_index()
stats.chisquare(obs, f_exp=50 * pr)  # statistic=0.3704, p=0.9463
```

Trier les modalités dans le même ordre

`chisquare` compare les cases **dans l'ordre** : observés et attendus doivent être triés de la même façon, d'où les deux `sort_index()`. `chi2_contingency` applique, comme R, la correction de Yates aux seuls tableaux 2×2.

La régression

```
import statsmodels.formula.api as smf
mod = smf.ols("salaire ~ anciennete", data=ech).fit()
pd.DataFrame({"coef": mod.params, "se": mod.bse,
              "t": mod.tvalues, "p": mod.pvalues}).round(4)
mod.conf_int().round(2)
print(mod.summary())    # tableau complet : R2, F, asymétrie des résidus
```

La sortie (des deux premières lignes)

```
                 coef        se       t    p
Intercept   4288.7182  468.5782  9.1526  0.0
anciennete   263.7943   39.5428  6.6711  0.0
                    0         1
Intercept   3346.58   5230.86
anciennete   184.29    343.30
```

Les prévisions

```
p = mod.get_prediction(pd.DataFrame({"anciennete": [10]}))
p.summary_frame(alpha=0.05)
```

La sortie, en colonnes

Colonne	Valeur	Colonne	Valeur
`mean`	6 926,66	`mean_se`	231,99
`mean_ci_lower`	6 460,21	`mean_ci_upper`	7 393,11
`obs_ci_lower`	3 599,81	`obs_ci_upper`	10 253,52

`mean` pour la moyenne, `obs` pour un individu

Les colonnes `mean_ci` donnent l'intervalle du salaire moyen ; les colonnes `obs_ci`, l'intervalle de prévision d'un salarié. Ce sont les nombres du chapitre 20, et ceux de `predict()` en R.

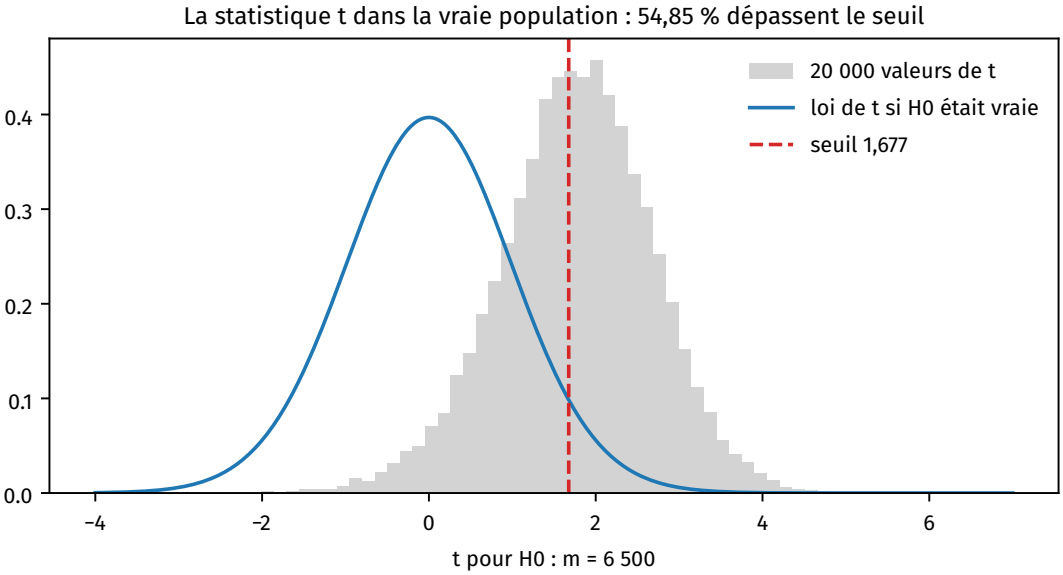
La simulation du chapitre 13, reproduite à l'identique

```
x = sal.salaire.to_numpy()
rng = np.random.default_rng(20261022)
A = x[rng.integers(0, 500, (20000, 50))]   # 20 000 échantillons
moy, s = A.mean(axis=1), A.std(axis=1, ddof=1)
T = (moy - 6500) / (s / np.sqrt(50))
np.mean(T > stats.t.ppf(0.95, 49))   # 0.5485
moy.std()                            # 368.37  (théorie : 367.43)
```

La même graine, les mêmes nombres

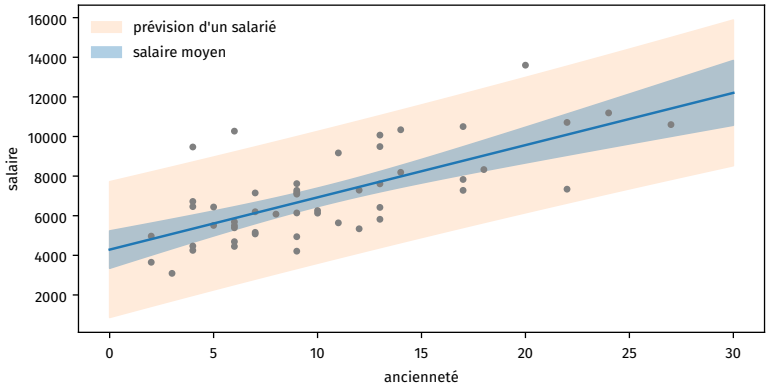
Toutes les simulations du cours ont été faites avec `numpy` et la graine `20261022` : ce bloc redonne exactement la puissance simulée du chapitre 13, 0,5485.
R utilise un autre générateur : avec la même graine, il donne d'autres échantillons, donc des résultats voisins mais pas identiques.

Le résultat, en image



Les deux bandes, en Python

```
grille = pd.DataFrame({"anciennete": np.linspace(0, 30, 61)})
p = mod.get_prediction(grille).summary_frame(alpha=0.05)
plt.fill_between(grille.anciennete, p.obs_ci_lower, p.obs_ci_upper)
plt.fill_between(grille.anciennete, p.mean_ci_lower, p.mean_ci_upper)
plt.scatter(ech.anciennete, ech.salaire)
```



Les pièges et la correspondance

Les réglages par défaut qui changent le résultat

Cinq différences à connaître

Calcul	R, par défaut	Python, par défaut
écart-type	`sd()` : $n-1$	`np.std()` : n ; `pandas` : $n-1$
deux moyennes	`t.test()` : Welch	`ttest_ind()` : variances égales
une proportion	`prop.test()` : avec correction	`proportions_ztest()` : f au dénominateur
khi-deux 2×2	correction de Yates	correction de Yates
simulations	générateur de R	générateur de `numpy`

La règle

Avant de faire confiance à une fonction, la vérifier **une fois** sur un exemple calculé à la main. C'est ce que ce chapitre a fait pour chacune.

La correspondance des fonctions

R et Python, test par test

Calcul	R	Python
test et IC d'une moyenne	`t.test`	`stats.ttest_1samp`
une proportion	`prop.test`, `binom.test`	`proportions_ztest`, `binomtest`
puissance, taille	`power.t.test`	`TTestPower`
deux groupes	`t.test(y ~ g)`	`stats.ttest_ind`, `ttest_rel`
ANOVA	`aov`, `oneway.test`	`stats.f_oneway`, `anova_oneway`
khi-deux	`chisq.test`	`chi2_contingency`, `chisquare`
régression	`lm`, `confint`, `predict`	`smf.ols`, `conf_int`, `get_prediction`

S'entraîner

Exercice 1 : lire une sortie R

Deux agences comparées

Un analyste compare les délais de traitement (en jours) de deux agences et obtient :

```
    Welch Two Sample t-test
t = -2.4512, df = 61.37, p-value = 0.01710
95 percent confidence interval:
 -3.1956 -0.3244
mean in group A mean in group B
          8.12           9.88
```

1. Quelle est l'hypothèse testée, et quelle est la conclusion au seuil de 5 % ?
2. Que représente l'intervalle ? Vérifier qu'il est cohérent avec les moyennes.
3. Au seuil de 1 %, la conclusion change-t-elle ?

Exercice 1 : solution

Chaque ligne a un sens

1. $H_0 : m_A = m_B$ contre $m_A \neq m_B$ (bilatéral par défaut). $p = 0{,}017 < 0{,}05$: les délais moyens diffèrent.
2. L'intervalle de $m_A - m_B$: l'agence A est plus rapide de 0,32 à 3,20 jours. Son milieu, $-1{,}76$, est bien $8{,}12 - 9{,}88$.
3. Oui : $0{,}017 > 0{,}01$, on ne rejetterait plus. Le verdict dépend du seuil, fixé à l'avance (chapitre 14).

Exercice 2 : corriger un script Python

Cinq erreurs

Le script doit tester, sur l'échantillon fil rouge, l'égalité des salaires hommes–femmes (Welch), puis la part de Bac+5 contre 0,15 (test du cours).

```
ech = sal[sal.id % 10 == 1]
s = np.std(ech.salaire)
h = ech[ech.sexe == "H"].salaire; f = ech[ech.sexe == "F"].salaire
stats.ttest_ind(h, f)
proportions_ztest(13, 50, value=0.15)
stats.chi2_contingency(pd.crosstab(ech.diplome, ech.sexe)).pvalue < 5
```

Trouver les cinq erreurs.

Exercice 2 : solution

Les corrections

1. `id % 10 == 1` sélectionne un autre échantillon : il faut `== 0`.
2. `np.std` divise par n : utiliser `ech.salaire.std()` (ou `ddof=1`).
3. `ttest_ind(h, f)` suppose des variances égales : ajouter `equal_var=False`.
4. `proportions_ztest` met f au dénominateur : ajouter `prop_var=0.15`.
5. Une p-value se compare à 0,05, pas à 5.

Exercice 3 : écrire le code

Une ANOVA complète

Sur l'échantillon fil rouge, on veut savoir si le salaire moyen diffère selon la **région** (quatre modalités).

1. Écrire les lignes R qui donnent le tableau d'analyse de la variance, la version de Welch et les comparaisons deux à deux corrigées.
2. Écrire les lignes Python équivalentes.
3. Combien de comparaisons deux à deux, et quel seuil corrigé ?

Exercice 3 : solution

Les mêmes trois étapes dans les deux langages

```r
summary(aov(salaire ~ region, data = ech))
oneway.test(salaire ~ region, data = ech)
pairwise.t.test(ech$salaire, ech$region,
                p.adjust.method = "bonferroni", pool.sd = FALSE)
```

```python
g = [x.salaire.to_numpy() for _, x in ech.groupby("region")]
stats.f_oneway(*g); anova_oneway(g, use_var="unequal")
```

Puis `stats.ttest_ind(..., equal_var=False)` sur chaque paire et `multipletests(p, method="bonferroni")`. Quatre régions : 6 comparaisons, seuil 0,05/6 = 0,0083. Résultat : $F = 1{,}24$, $p = 0{,}31$ (Welch 0,29) : aucun effet détectable de la région.

Ce qu'il faut retenir

Le résumé du chapitre

Six points

1. `t.test`, `prop.test`, `aov`, `chisq.test`, `lm` : cinq fonctions de R couvrent l'essentiel du cours ; `scipy.stats` et `statsmodels` font de même en Python.
2. `t.test()` donne à la fois le test **et** l'intervalle — les deux faces du chapitre 15.
3. Les réglages par défaut diffèrent : Welch en R, variances égales dans `ttest_ind` ; p_0 ou f au dénominateur ; correction de continuité.
4. `pandas.std()` divise par $n-1$, `numpy.std()` par n.
5. Toute la sortie de `lm` et de `ols` a été calculée à la main aux chapitres 19 et 20.
6. Quelques lignes de simulation **vérifient** la loi de la moyenne, la puissance et la couverture des intervalles.

La fin du parcours

Une seule idée

Nous sommes partis de cinq cents salaires que l'on se contentait de décrire. Nous arrivons à un modèle qui les explique, avec la mesure de sa fiabilité.
Entre les deux, une seule idée aura tout porté : **une statistique calculée sur un échantillon est une variable aléatoire, et l'on connaît sa loi**. Estimer, encadrer, tester, régresser n'en sont que quatre usages.

La suite

L'introduction à l'économétrie prend la suite immédiate : mêmes outils, plusieurs variables explicatives, et l'examen systématique des hypothèses du chapitre 19.